

CTTP: 여러 round로 triangle 열거하기

MapReduce Triangle Enumeration With Guarantees

Ha-Myung Park, Francesco Silvestri, U Kang, Rasmus Pagh (2014)

색 기반 subproblem을 분산해 한 round의 intermediate-data peak를 제한한다

Triangle enumeration의 출력 경계

입력은 simple undirected graph이며 triangle은 서로 다른 세 vertex와 세 edge로 구성된다

- | | | |
|----|------------|---|
| 01 | Graph edge | 중복 edge와 self-loop가 없는 undirected graph를 입력받는다 |
| ↓ | | |
| 02 | 세 vertex | 서로 다른 u, v, w 를 후보로 본다 |
| ↓ | | |
| 03 | 세 edge 확인 | uv, uw, vw 가 모두 실제 graph에 있어야 한다 |
| ↓ | | |
| 04 | Local enum | 찾은 triangle마다 $\text{enum}(u, v, w)$ 를 호출할 수 있다 |

논문 모델은 모든 triangle record를 하나의 거대한 파일로 저장하도록 요구하지 않는다

원본 graph보다 커지는 shuffle

한 input edge가 여러 triangle subproblem에 필요하면 mapper가 같은 edge record를 여러 key로 복제한다

01	Input edge	mapper가 한 edge record를 읽는다
↓		
02	여러 subproblem	그 edge가 필요한 reducer key를 계산한다
↓		
03	Record 복제	같은 edge를 여러 key-value record로 emit한다
↓		
04	Shuffle regroup	system이 key별로 모든 복제본을 동시에 모은다
↓		
05	One-round peak	reducer 실행 전에 aggregate data가 capacity를 넘을 수 있다

분산 저장된 graph도 intermediate edge copies가 몰리면 한 round를 실행하지 못할 수 있다

MR(m, M)의 resource limits

E는 graph edge 수이며 m과 M은 서로 다른 scope의 memory-space bound다

기호	정의	초과하면 생기는 문제
E	입력 edge 수	Partition·round 식의 기준
m	함수 하나의 최대 words	Key group이 크면 실패
M	한 round의 총 words	전체 group 합이 크면 실패
m _{k,r}	Key k의 round-r words	m _{k,r} ≤ m 필요
Aggregate	같은 round의 m _{k,r} 합	합계 ≤ M 필요

m은 한 함수의 limit이고 M은 cluster 전체의 one-round aggregate limit이다

색은 vertex identity가 아니라 partition label

ρ 는 color 수이고 $\xi(v)$ 는 vertex v 에 배정한 color label이다

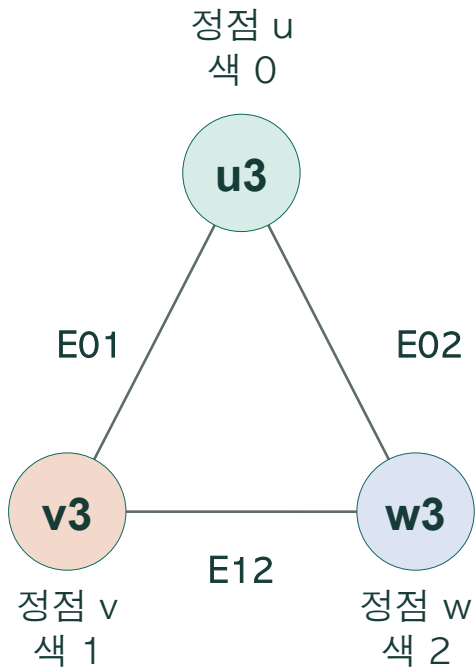
표현	뜻	주의할 점
ρ	Color label 수	$\text{ceil}(\sqrt{6E/m})$
$\xi(v)$	v 의 random label	Vertex 자체는 그대로
$E_{(i,j)}$	Label pair i,j 의 edge	$i \leq j$ 로 기록
같은 color	같은 bucket label	Vertex 합체가 아님
인접 vertex	같은 label도 가능	Proper coloring 아님

Color label은 triangle을 어느 작은 task에 보낼지 결정하며 original edge는 그대로다

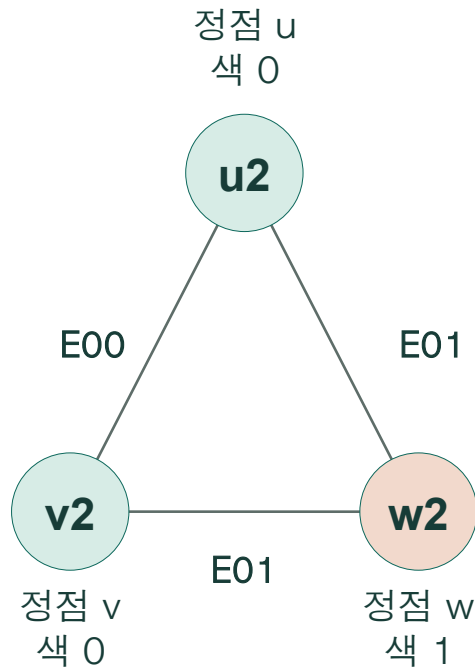
Color label로 정한 세 triangle type

같은 original triangle도 x_i 를 다시 선택하면 다른 type에 속할 수 있다

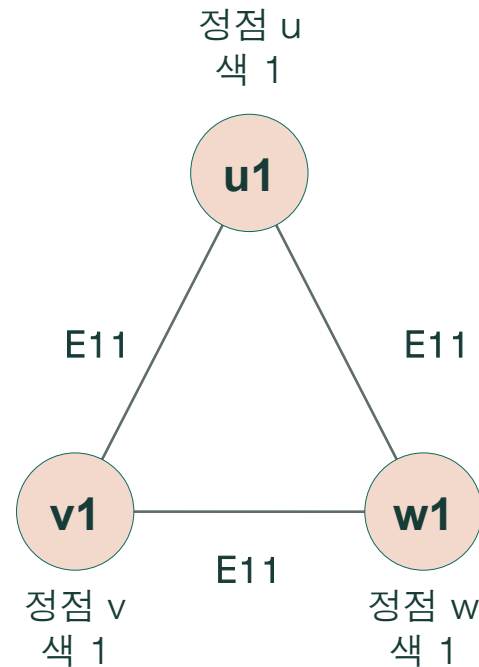
Type-3 · 색 라벨 0,1,2



Type-2 · 색 라벨 0,0,1

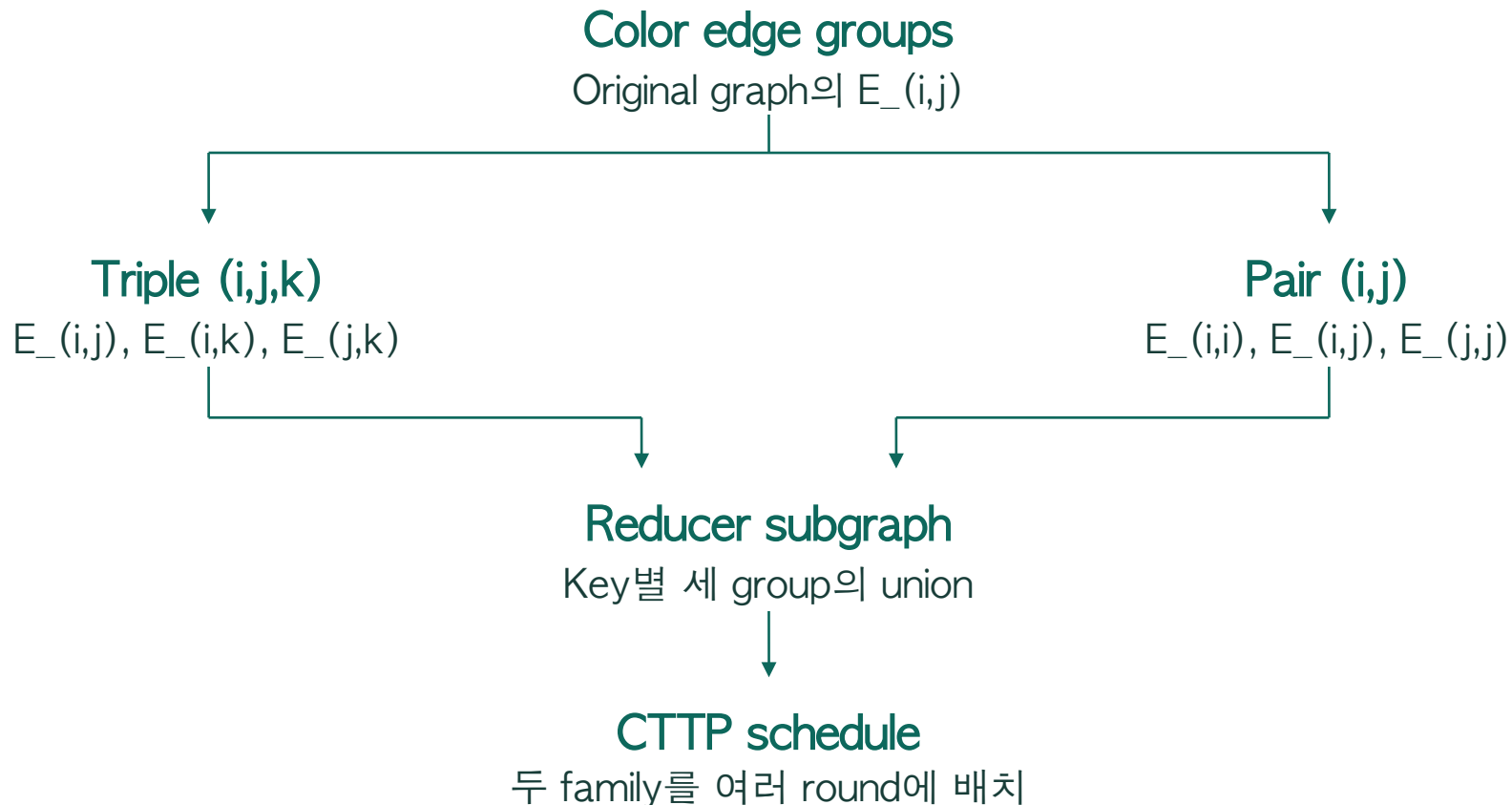


Type-1 · 색 라벨 1,1,1



TTP와 CTP가 사용하는 두 task family

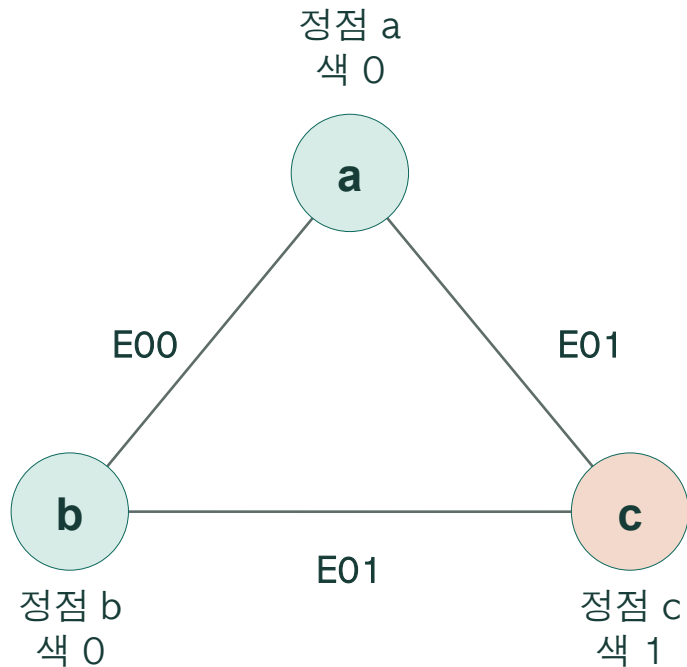
두 family가 필요한 edge groups를 모은 뒤 같은 local triangle enumerator를 실행한다



교육용 예시: Type-2 triangle의 routing

서로 다른 a, b, c의 색이 0,0,1이면 E00-E01 groups가 pair reducer (0,1)로 모인다

Pair task (0,1): E00-E01 $\rightarrow$ reducer



E00
a-b

E01
a-c, b-c

교육용 예시: rho=4의 edge routing

Color는 0, 1, 2, 3이며 한 edge가 필요한 TTP-style task를 정확히 센다

대상	필요한 task keys	Copy 수	의미
Internal E_(0,0)	(0,1), (0,2), (0,3)	3	rho- 1 pair tasks
Cross E_(0,1)	(0,1), (0,1,2), (0,1,3)	3	1 pair + rho-2 triples
Type-1 (0,0,0)	(0,1), (0,2), (0,3)에 존재	3	Coverage와 1회 계산은 다름

CTTP는 rho- 1 copies를 없애지 않고 서로 다른 round에 나누어 보낸다

GP와 TTP의 constant-factor 차이

아래 값은 edge가 color pair에 고르게 분포한다는 계산에서 얻은 expected task input이다

방법과 task	Expected edge input	변경점
GP three-color	$9E/\rho^2$	Internal과 cross edge case를 함께 포함
TTP type-3	$6E/\rho^2$	필요한 세 cross-color groups만 사용
TTP type- 1/2	$4E/\rho^2$	두 internal과 한 cross group 사용
CTTP	TTP와 같은 task input	같은 task를 여러 round에 schedule

9, 6, 4는 constant coefficient이며 asymptotic class나 실제 runtime 비율을 바꾸지 않는다

CTTP가 유지한 것과 추가한 것

R은 전체 MapReduce round 수이며 base coloring은 4-wise independent family를 사용한다

항목	TTP에서 유지	CTTP에서 추가
Color subproblem	Triple (i,j,k) , pair (i,j)	같은 family를 round에 배정
Color 수 ρ	Expected task size를 m 에 맞춤	$\rho = \text{ceil}(\sqrt{6E/m})$
Round 수 R	One-round partition	$R = \rho E/M$ 규모로 분산
Coloring analysis	Type-based partition	Base는 4-wise independent x_i
System 목표	Reducer-local enumeration	One-round aggregate를 M 이하로 제한

CTTP의 새 요소는 triangle type이 아니라 multi-round scheduling과 load guarantee다

Subproblem의 modular round assignment

현재 round를 r 이라고 할 때 각 logical subproblem은 color-index sum의 residue로 한 round에
배정된다

Task family	Round 조건	Reducer가 받는 groups
Triple (i,j,k)	$i + j + k = r \bmod R$	$E_{-}(i,j), E_{-}(i,k), E_{-}(j,k)$
Pair (i,j)	$i + j = r \bmod R$	$E_{-}(i,i), E_{-}(i,j), E_{-}(j,j)$
Input edge	현재 r 에 맞는 key만 emit	다른 key는 해당 round까지 기다림
전체 R rounds	모든 residue를 한 번씩 실행	필요한 logical task를 모두 cover

Schedule은 task 실행 시점을 바꾸며 triangle type이나 필요한 edge group을 바꾸지 않는다

교육용 예시: $\rho=4$, $R=2$ 의 round 분산

$E_{-}(0,1)$ edge의 세 logical copies를 task-key sum mod 2로 정확히 배치한다

Task key	Index sum	Assigned round	해당 edge copy
Pair (0,1)	$0 + 1 = 1$	$r = 1$	$E_{-}(0,1)$
Triple (0,1,2)	$0 + 1 + 2 = 3$	$r = 1$	$E_{-}(0,1)$
Triple (0,1,3)	$0 + 1 + 3 = 4$	$r = 0$	$E_{-}(0,1)$
Round 0 합계	한 task	한 copy	전체 세 copy 중 1
Round 1 합계	두 tasks	두 copies	전체 세 copy 중 2

한 round의 copy 수는 줄지만 두 round 뒤에는 필요한 세 deliveries가 모두 일어난다

한 edge가 현재 round를 통과하는 과정

Mapper는 현재 round에 schedule된 key만 만들고 reducer는 필요한 세 edge group의 union을 받는다

01	Map input	Edge (u,v) 를 읽고 $x_i(u)$, $x_i(v)$ 를 계산한다
↓		
02	Group 확인	Ordered color pair로 $E_{-}(i,j)$ 를 정한다
↓		
03	Current-round emit	Color sum이 현재 r 과 맞는 task key만 emit한다
↓		
04	Shuffle	같은 pair 또는 triple key의 edge records를 모은다
↓		
05	Reduce와 enum	세 group의 union에서 triangle을 local enumerate한다

다음 round에는 input을 다시 읽어 그 round의 다른 task keys를 생성한다

Round 수와 data movement의 tradeoff

Per-round shuffle은 $(\rho - 1)E/R$ 이며 R을 늘리면 한 번의 peak가 낮아진다

항목	R이 작을 때	R이 클 때
한 round의 shuffle	많은 edge copies가 동시에 활성화	더 적은 copies가 활성화
Aggregate peak	M을 넘을 위험이 큼	M에 맞추기 쉬움
Input scan	적은 횟수	매 round에서 다시 읽음
Startup과 map cost	적은 반복	Round 수만큼 반복
전체 task deliveries	필요한 TTP copies	여전히 필요한 TTP copies

CTTP는 total work를 무료로 제거하지 않고 one-round peak와 repeated overhead를 교환한다

Theorem 1 : Base CTPP의 expected guarantee

Base 4-wise-independent coloring에서 reducer load와 total work는 expectation으로 제시된다

보장 항목	Theorem 1의 bound	Guarantee 수준
Round 수	$O(E^{1.5} / (M \sqrt{m}))$	Asymptotic bound
한 mapper space	M/E	Construction bound
한 reducer space	m	Expected
한 round aggregate	M	Bound
Total work	$O(E^{1.5})$	Expected

Expected reducer size는 모든 random coloring에서 deterministic $O(m)$ 을 뜻하지 않는다

Theorem 2의 worst-case round lower bound

m_i 는 reducer i 가 받은 words이며 모든 reducer 합은 M 이하라고 둔다

01	한 reducer	m_i words로 최대 $m_i^{1.5}$ 개 distinct triangles를 열거한다
↓		
02	Local limit	각 m_i 는 한 함수 limit m 이하이다
↓		
03	Aggregate limit	같은 round의 모든 m_i 합은 M 이하이다
↓		
04	One-round output	한 round에서 최대 $M \sqrt{m}$ triangles를 열거한다
↓		
05	Lower bound	t triangles에는 $\Omega(t / (M \sqrt{m}))$ rounds가 필요하다

Optimal은 $MR(m, M)$ 와 memory-word assumption 안의 asymptotic worst-case 의미다

Expected load와 last-reducer risk

전체 평균이 작아도 특정 color group에 edge가 몰리면 가장 늦은 reducer가 round를 지연시킬 수 있다

관점	Base expected result	남는 위험
한 reducer	평균적으로 m 규모	특정 key가 m보다 클 수 있음
Round 완료	대부분 reducer가 작을 수 있음	가장 큰 reducer를 기다려야 함
4-wise coloring	Expected analysis를 지지	모든 load의 deterministic bound는 아님
필요한 강화	Coloring과 degree 처리를 수정	High-probability maximum load bound

Curse of the last reducer는 평균이 아니라 maximum reducer input의 불균형 문제다

Theorem 3: 추가 조건 아래의 high-probability guarantee

Full variant는 very-high-degree preprocessing과 degree-aware coloring을 함께 사용한다

구성	조건 또는 방법	역할
Very-high degree	$\text{degree} \geq \sqrt{E m}$: sorting	Main range 분리
Low degree	$\text{degree} \leq \sqrt{E}$: log-E-wise hash	Load 집중
High degree	$\sqrt{E} < \text{degree} \leq \sqrt{E m}$	Degree-mass coloring
Memory condition	$m = \Omega(E^{0.75} \sqrt{\log E})$	적용 조건
Probability	at least $1 - 1/E^\epsilon$	$O(m)$; work $O(E^{1.5})$

강한 bound에는 전처리, coloring 변경, memory 조건이 모두 필요하다

2014 Hadoop experiment setup

아래 결과는 논문이 보고한 Java/Hadoop implementation과 KAIST cluster 조건의 evidence다

항목	보고된 설정	해석 범위
Implementation	Java와 Hadoop	논문의 세 algorithm을 비교
Cluster	40 machines	당시 실험 infrastructure
Memory	각 machine 4 GB	총 RAM이나 M과 동일시하지 않음
Reducers	Default 80	Experiment parameter
시점	CIKM 2014	현재 framework 성능 claim이 아님

Failure threshold와 runtime은 이 구현·cluster·dataset에 한정된다

Twitter에서 관찰한 one-round shuffle peak

보고된 2014 setup에서 $R=1$ 은 32 GB input으로부터 985 GB shuffle을 생성했다

측정 항목	보고값	단위	의미
Twitter input	32	GB	원본 input size
$R=1$ shuffle	985	GB	한 round의 intermediate data
크기 비	약 30.8	배	$985 / 32$ 의 단순 비율
$R < 4$	Unexpected high runtime	관찰	Map과 shuffle overload로 설명
R 증가	Per-round shuffle 감소	Tradeoff	Input read와 startup 반복

Graph file이 32 GB여도 같은 edge copies가 몰린 shuffle은 985 GB에 이를 수 있었다

YahooWeb subgraph의 data scalability 결과

같은 2014 Hadoop setup에서 random YahooWeb subgraph의 edge 수를 늘려 세 방법을 비교했다

Edge 수	CTTP	TTP와 GP	해석
0.2B- 1.6B	Reported runs 완료	Reported runs 완료	세 방법이 비교 가능한 구간
3.2B	완료	Out-of-space failure	1.6B를 넘은 reported point
6.4B	완료	Out-of-space failure	가장 큰 reported subgraph
Twitter machine scaling	세 방법 중 ideal에 가장 가까움	상대적으로 낮은 scaling	동일 구현의 machine-scaling 관찰

1.6B 이후 failure는 reported cluster 결과이며 hardware-independent limit가 아니다

Real graph의 reported enumeration time

단위는 minutes이며 failure는 같은 2014 Hadoop setup에서의 결과다

Dataset	CTTP	TTP	GP	단위
LiveJournal	1	1	1	min
PhoneCall	6	6	11	min
Twitter	92	162	302	min
SubDomain	218	455	1496	min
ClueWeb09	1328	Failed	Failed	min

ClueWeb09의 31,013,037,486 triangles를 CTTP가 약 23 hours에 열거했다고 보고했다

Later PTE paper와의 관계

이 비교는 후속 연구 맥락이며 두 paper의 동일 구현 benchmark가 아니다

관점	CTTP 2014	PTE 2016 context
핵심 문제	One-round shuffle peak	Massive intermediate data
CTTP의 선택	TTP tasks를 round에 분산	Related-work baseline
남는 비용	Input reread와 task copies	다른 read pipeline
Type-1	여러 pair task에 존재	중복 계산을 지적
비교 한계	CIKM 2014 setup	KDD 2016 별도 setup

CTTP의 multi-round scheduling과 PTE의 later redesign을 같은 algorithm으로 합치면 안 된다

CTTP의 핵심과 주장 범위

TTP의 color-type partition을 여러 round에 schedule해 one-round aggregate space를 통제한다

구분	논문이 제시한 내용	제시하지 않은 내용
Mechanism	Pair-triple의 modular schedule	Replication 완전 제거
Base guarantee	Expected m ; work $O(E^{1.5})$	Deterministic load
Strong guarantee	Degree-memory 조건부 high probability	무조건적 load balance
Experiment	2014 Hadoop evidence	현대 framework 일반화
Output	Reducer-local enum	전체 record 저장 의무

CTTP는 coverage를 유지하며 한 round의 intermediate data를 제한한다